==
INTERLEAVED 1F1B PIPELINE PARALLELISM EXPLAINED
==

TOPIC 1: Understanding Interleaved Stages - Dark Green vs Light Green
==

QUESTION:
In the interleaved pipeline diagram:
1. GPU1 has: dark green 1 (forward pass first layer), light green 1 (forward pass last layer)
2. GPU4 has: dark green 1 (forward pass first layer), light green 1 (forward pass last layer)

How does each GPU process both "first layer" and "last layer" of the same micro-batch?

ANSWER:

The Key Insight: Virtual Pipeline Stages

In interleaved pipeline parallelism, each physical GPU is responsible for MULTIPLE
NON-CONTIGUOUS chunks of the model layers. This is what "interleaved" means.

Concrete Example with 24-layer Model:

Total layers: 24
GPUs: 4
Virtual stages per GPU (v): 2

Layer Distribution (INTERLEAVED pattern):

GPU1 owns:
  - Chunk 1 (dark green): Layers 0-2   ← "first layer" chunk
  - Chunk 2 (light green): Layers 12-14 ← "last layer" chunk

GPU2 owns:
  - Chunk 1 (dark green): Layers 3-5
  - Chunk 2 (light green): Layers 15-17

GPU3 owns:
  - Chunk 1 (dark green): Layers 6-8
  - Chunk 2 (light green): Layers 18-20

GPU4 owns:

```
  - Chunk 1 (dark green): Layers 9-11
  - Chunk 2 (light green): Layers 21-23

Data Flow for Micro-batch 1:
----------------------------

Phase 1 - First Chunks (Dark Green):
  Step 1: MB-1 → GPU1[layers 0-2]    → output to GPU2
  Step 2: MB-1 → GPU2[layers 3-5]    → output to GPU3
  Step 3: MB-1 → GPU3[layers 6-8]    → output to GPU4
  Step 4: MB-1 → GPU4[layers 9-11]   → output back to GPU1

Phase 2 - Second Chunks (Light Green):
  Step 5: MB-1 → GPU1[layers 12-14]  → output to GPU2
  Step 6: MB-1 → GPU2[layers 15-17]  → output to GPU3
  Step 7: MB-1 → GPU3[layers 18-20]  → output to GPU4
  Step 8: MB-1 → GPU4[layers 21-23]  → final output

Why "First Layer" and "Last Layer" on Each GPU?
-----------------------------------------------
- "First layer" (dark green) = The FIRST chunk this GPU owns in the layer
sequence
  - GPU1's first chunk: layers 0-2 (entry point of model)
  - GPU4's first chunk: layers 9-11 (middle of model)

- "Last layer" (light green) = The SECOND chunk this GPU owns in the
layer sequence
  - GPU1's last chunk: layers 12-14 (middle of model)
  - GPU4's last chunk: layers 21-23 (exit point of model)

Each micro-batch makes TWO PASSES through each GPU:
  1st pass: Through the GPU's "first" chunk (dark green)
  2nd pass: Through the GPU's "last" chunk (light green)

Timeline for GPU1 Processing Micro-batch 1:
-------------------------------------------
Time T1: Process MB-1 through layers 0-2 (dark green 1)
    ↓ MB-1 flows through GPU2, GPU3, GPU4's first chunks
Time T5: Process MB-1 through layers 12-14 (light green 1)
    ↓ MB-1 flows through GPU2, GPU3, GPU4's second chunks
Time T8: MB-1 completes entire model

Why Interleaving Reduces Bubble?
--------------------------------
Without interleaving (v=1):
  - Each GPU processes ONE LARGE chunk (e.g., 6 layers)
  - GPU4 waits longer for MB-1 to traverse GPU1→2→3

With interleaving (v=2):
  - Each GPU processes TWO SMALLER chunks (e.g., 3 layers each)
  - Pipeline fills faster because each chunk is smaller
  - GPU4 starts working on MB-1's first chunk sooner
```

```
================================================================================
========
TOPIC 2: Deriving the 1F1B Interleaving Formulas
================================================================================
========

Variables Definition:
---------------------
p  = number of GPUs (pipeline stages) = 4
v  = number of virtual stages (model chunks) per GPU = 2
m  = number of micro-batches = 16
t_f = time for one forward pass through ONE CHUNK
t_b = time for one backward pass through ONE CHUNK

FORMULA 1: Pipeline Bubble Time
================================================================================
========

t_pb = (p - 1) × (t_f + t_b) / v

Step-by-Step Derivation:
------------------------

Step 1: Understand Pipeline Startup Delay
-----------------------------------------
In any pipeline, there's an initial "bubble" where not all GPUs are
working.

Example: 4 GPUs, first micro-batch startup:

Time Step | GPU1      | GPU2      | GPU3      | GPU4
----------|-----------|-----------|-----------|----------
    1     | Working   | IDLE      | IDLE      | IDLE
    2     | Working   | Working   | IDLE      | IDLE
    3     | Working   | Working   | Working   | IDLE
    4     | Working   | Working   | Working   | Working ← Pipeline full

Idle time slots = (p-1) = 3 GPU-steps
```

Step 2: Calculate Bubble for Non-Interleaved Pipeline (v=1)

Each GPU processes the ENTIRE model chunk in one shot.
If each chunk takes (t_f + t_b) time for forward+backward:

Bubble = (p-1) × (t_f + t_b)
       = (4-1) × (t_f + t_b)
       = 3 × (t_f + t_b)

Example with numbers:
  If t_f = 100ms, t_b = 100ms
  Bubble = 3 × 200ms = 600ms

Step 3: Calculate Bubble for Interleaved Pipeline (v=2)

Each GPU processes v=2 smaller chunks instead of 1 large chunk.
Each small chunk takes: (t_f + t_b) / v time

The startup delay is still (p-1) steps, but each step is SHORTER:

Bubble = (p-1) × [(t_f + t_b) / v]
       = (4-1) × [(t_f + t_b) / 2]
       = 3 × (t_f + t_b) / 2
       = 1.5 × (t_f + t_b)

Example with numbers:
  If t_f = 100ms, t_b = 100ms (for the FULL chunk)
  Each small chunk: 50ms forward + 50ms backward = 100ms total
  Bubble = 3 × 100ms = 300ms

Result: Interleaving CUTS BUBBLE IN HALF!

FORMULA 2: Bubble Ratio
==========
========

r_bubble = (p - 1) / (v × m)

Step-by-Step Derivation:

Step 1: Calculate Total Computation Time

We have m micro-batches, each requires forward+backward through ALL
layers:

Total_time = m × (t_f + t_b)

Example:
  m = 16 micro-batches
  t_f + t_b = 200ms per full model pass
  Total_time = 16 × 200ms = 3200ms

```
Step 2: Calculate Bubble Ratio
------------------------------
Bubble ratio = Bubble_time / Total_time

Substitute Formula 1:
  r_bubble = [(p-1) × (t_f + t_b) / v] / [m × (t_f + t_b)]

Simplify (cancel out t_f + t_b):
  r_bubble = (p-1) / (v × m)

Step 3: Verify with Numbers
---------------------------
Case 1: WITHOUT interleaving (v=1)
  p = 4, v = 1, m = 16
  r_bubble = (4-1) / (1 × 16) = 3/16 = 0.1875 = 18.75%

  Verification:
    Bubble = 3 × (t_f + t_b) = 3 × 200ms = 600ms
    Total = 16 × 200ms = 3200ms
    Ratio = 600 / 3200 = 0.1875 ✓

Case 2: WITH interleaving (v=2)
  p = 4, v = 2, m = 16
  r_bubble = (4-1) / (2 × 16) = 3/32 = 0.09375 = 9.375%

  Verification:
    Bubble = 3 × (t_f + t_b) / 2 = 3 × 200ms / 2 = 300ms
    Total = 16 × 200ms = 3200ms
    Ratio = 300 / 3200 = 0.09375 ✓

Conclusion: Interleaving with v=2 cuts bubble overhead from 18.75% to
9.375%


VISUAL TIMELINE EXAMPLE - CORRECTED
================================================================================
========

Setup: p=4 GPUs, v=2 virtual stages, processing 3 micro-batches

Legend:
  F1a = Forward micro-batch 1, chunk a (GPU's first chunk - dark green)
  F1b = Forward micro-batch 1, chunk b (GPU's second chunk - light green)
  B1a = Backward micro-batch 1, chunk a
  B1b = Backward micro-batch 1, chunk b
  BUBBLE = Idle time

CRITICAL UNDERSTANDING:
-----------------------
Forward flow: MB goes through chunks in LAYER ORDER
  F1a: GPU1[0-2] → GPU2[3-5] → GPU3[6-8] → GPU4[9-11]
  F1b: GPU1[12-14] → GPU2[15-17] → GPU3[18-20] → GPU4[21-23]
```

Backward flow: REVERSE layer order (backpropagation flows backwards!)
  B1b: GPU4[21-23] → GPU3[18-20] → GPU2[15-17] → GPU1[12-14]
  B1a: GPU4[9-11] → GPU3[6-8] → GPU2[3-5] → GPU1[0-2]

CORRECTED TIMELINE:

Time	GPU1	GPU2	GPU3	GPU4	Notes
1	F1a	BUBBLE	BUBBLE	BUBBLE	MB1 enters pipeline
2	F2a	F1a	BUBBLE	BUBBLE	MB2 enters
3	F3a	F2a	F1a	BUBBLE	MB3 enters
4	F1b	F3a	F2a	F1a	GPU4 gets first work
5	F2b	F1b	F3a	F2a	
6	F3b	F2b	F1b	F3a	
7	BUBBLE	F3b	F2b	F1b	MB1 F1b processing (not done yet!)
8	BUBBLE	BUBBLE	F3b	F2b	MB1 forward DONE! Can start B1b
9	BUBBLE	BUBBLE	BUBBLE	F3b	
10	BUBBLE	BUBBLE	B1b	B1b	B1b STARTS on GPU4! ←
11	BUBBLE	B1b	B2b	B2b	B2b starts (F2b done at time 8)
12	B1b	B2b	B3b	B3b	B1b reaches GPU1
13	B2a	B3b	B1a	B1a	B1a STARTS on GPU4! ←
14	B1a	B1a	B2a	B2a	B1a flows backwards
15	B3a	B2a	B3a	B3a	
16	B3b	B3a	BUBBLE	BUBBLE	
17	Done	BUBBLE	BUBBLE	BUBBLE	All done

CRITICAL: WHY B1b STARTS BEFORE B1a
==========

Q1: "B1a has not started, how come B1b started before B1a?"

ANSWER: Backpropagation MUST flow in REVERSE layer order!
  - B1b contains layers 21-23 (HIGHEST layers - output end)
  - B1a contains layers 9-11 (LOWER layers - middle of network)
  - Backprop flows: output → input (layers 23 → 0)
  - Therefore: B1b MUST happen before B1a

This is fundamental to backpropagation: gradients flow backwards from loss!

Q2: "Why does B1b start at time 10, not earlier?"

ANSWER: B1b can only start AFTER F1b completes AND GPU4 is free!

Time 7: GPU4 is PROCESSING F1b (not done yet!)
Time 8: F1b completes at END of time 7, GPU4 now processes F2b
Time 9: GPU4 processes F3b
Time 10: GPU4 is free, B1b can start (MB1 forward was done at time 8)

DETAILED TRACE FOR MICRO-BATCH 1:
==========

IMPORTANT: Each time slot represents one chunk computation (either F or B)

Time	Action	What's Happening
1	F1a on GPU1	GPU1 processes layers 0-2
2	F1a moves GPU1→GPU2	GPU2 processes layers 3-5
3	F1a moves GPU2→GPU3	GPU3 processes layers 6-8
4	F1a moves GPU3→GPU4	GPU4 processes layers 9-11
	F1b starts on GPU1	GPU1 processes layers 12-14
5	F1b moves GPU1→GPU2	GPU2 processes layers 15-17
6	F1b moves GPU2→GPU3	GPU3 processes layers 18-20
7	F1b PROCESSING on GPU4	GPU4 is BUSY processing layers 21-23
8	F1b COMPLETES on GPU4	← MB1 FORWARD DONE! Loss calculated!
	F2b now processing on GPU4	GPU4 switches to F2b
10	B1b can START on GPU4	GPU4 backprop through layers 21-23 ←
11	B1b moves GPU4→GPU3	GPU3 backprop through layers 18-20
12	B1b moves GPU3→GPU2	GPU2 backprop through layers 15-17
13	B1b moves GPU2→GPU1	GPU1 backprop through layers 12-14
		← B1b (chunk b) COMPLETE!
13	B1a STARTS on GPU4	GPU4 backprop through layers 9-11 ←
14	B1a moves GPU4→GPU3	GPU3 backprop through layers 6-8
15	B1a moves GPU3→GPU2	GPU2 backprop through layers 3-5
16	B1a moves GPU2→GPU1	GPU1 backprop through layers 0-2
		← MB1 BACKWARD COMPLETE!

Gradients ready.

KEY TIMING INSIGHTS:
==========

1. At time 7: GPU4 is BUSY executing F1b (it takes the entire time slot)

   - F1b is NOT yet complete during time 7
   - At the END of time 7, F1b finishes

2. At time 8: GPU4 is free and can do next task
   - Could do B1b (backward of MB1)
   - But 1F1B schedule says: continue with F2b first
   - So GPU4 processes F2b

3. Why B1b before B1a?
   - Backprop MUST flow backwards: layers 23→22→...→1→0
   - B1b = layers 21-23 (NEAR OUTPUT)
   - B1a = layers 9-11 (MIDDLE OF NETWORK)
   - Cannot do B1a before B1b - violates backprop!

4. Why not start B1b at time 7?
   - Time 7: GPU4 is PROCESSING F1b (not idle!)
   - Cannot do two things at once
   - Must wait until F1b completes

WHY B1a STARTS ON GPU4 (NOT GPU1):
==================================

The confusion comes from chunk naming vs. execution order:

CHUNK NAMING (relative to each GPU):
- "chunk a" = GPU's FIRST chunk in layer sequence
- "chunk b" = GPU's SECOND chunk in layer sequence

GPU1: chunk a = layers 0-2, chunk b = layers 12-14
GPU4: chunk a = layers 9-11, chunk b = layers 21-23

EXECUTION ORDER (backward flows in REVERSE layer order):
- MB1 forward: layers 0→2→...→21→23 (GPU1a→GPU2a→...→GPU4b)
- MB1 backward: layers 23→21→...→2→0 (GPU4b→GPU3b→...→GPU1a)

So B1a execution order: GPU4[9-11] → GPU3[6-8] → GPU2[3-5] → GPU1[0-2]
   B1b execution order: GPU4[21-23] → GPU3[18-20] → GPU2[15-17] → GPU1[12-14]

Backward processes chunk 'b' FIRST (higher layers), then chunk 'a' (lower layers)!

ANSWERING THE SPECIFIC QUESTIONS:
=================================

Q1: "B1a has not started, how come B1b started before B1a?"

A1: This is CORRECT behavior! B1b MUST happen before B1a because:
    - Backpropagation flows from OUTPUT to INPUT (reverse layer order)
    - B1b processes layers 21-23 (near the output/loss)
    - B1a processes layers 9-11 (middle of network)
    - Gradients must flow: layer 23 → 22 → ... → 1 → 0
    - Therefore: B1b (layers 21-23) before B1a (layers 9-11)

You CANNOT do B1a before B1b - this would violate backpropagation!

Q2: "If B1a supposed to start before B1b, why not B1a start at time 7?"

A2: This question has two issues:

Issue 1: B1a is NOT supposed to start before B1b!
- As explained in A1, B1b must come first (it's closer to output)
- B1a comes later (it's in the middle of the network)

Issue 2: Why doesn't B1b start at time 7?
- At time 7: GPU4 is BUSY processing F1b (computing layers 21-23 forward)
- A GPU can only do ONE thing per time slot
- F1b takes the entire time slot 7
- Only at time 8 (after F1b completes) can GPU4 do something else
- According to 1F1B schedule, GPU4 does F2b first (time 8-9)
- Then B1b can start at time 10

VISUALIZATION OF BACKWARD ORDER:
==========

Forward (increasing layer numbers):
GPU1[0-2] → GPU2[3-5] → GPU3[6-8] → GPU4[9-11] →
GPU1[12-14] → GPU2[15-17] → GPU3[18-20] → GPU4[21-23]
↓ Loss calculated here
Backward (DECREASING layer numbers):
GPU4[21-23] → GPU3[18-20] → GPU2[15-17] → GPU1[12-14] →
GPU4[9-11] → GPU3[6-8] → GPU2[3-5] → GPU1[0-2]
↑ B1a starts here (after B1b completes!)

Bubble analysis:
- GPU4 idle for first 3 time steps = (p-1) = 3 steps
- Each step processes one chunk (forward or backward)
- Total bubble = 3 × (time_per_chunk) = (p-1) × (t_f+t_b) / v ✓

==========
TOPIC 3: DECOMPOSING BACKWARD PASS (B vs W) - ADVANCED OPTIMIZATION
==========

::: Code Generated by Copilot [a8f2e4d7-9c3a-4f21-b89f-3e7d5a2b1c9f].
This comment will be removed automatically after the file is saved :::

KEY INSIGHT: Backward pass can be split into TWO independent operations:

```
================================================================================
=

Standard view:     Backward = compute gradients for everything
Optimized view:    Backward = B (activation gradients) + W (weight
gradients)

WHAT IS "B" (Backward for Activations):
=======================================
- Computes ∇_y L (gradient with respect to layer OUTPUT/activations)
- Required by PREVIOUS layer to continue backpropagation
- MUST be done immediately to unblock previous layers
- This is the "critical path" in backpropagation

WHAT IS "W" (Backward for Weights):
===================================
- Computes ∇_W L (gradient with respect to WEIGHTS)
- Only needed before optimizer.step() to update weights
- Does NOT block other layers from computing their gradients
- Can be delayed/scheduled flexibly to fill bubbles!

CONCRETE EXAMPLE - 3 Layer Network:
===================================

Network: Input → Layer1 → Layer2 → Layer3 → Loss

Forward:
  Layer1: z1 = W1 @ x,     y1 = σ(z1)
  Layer2: z2 = W2 @ y1,    y2 = σ(z2)
  Layer3: z3 = W3 @ y2,    y3 = σ(z3)
  Loss:   L = loss(y3, target)

Backward (DEPENDENCIES):

  At Layer3:
    ∇_y3 L = dL/dy3                        [given from loss]
    ∇_z3 L = ∇_y3 L * σ'(z3)               [chain rule]

    B3: ∇_y2 L = W3^T @ ∇_z3 L             [NEEDED by Layer2 - CRITICAL
PATH]
    W3: ∇_W3 L = ∇_z3 L @ y2^T             [only for optimizer - CAN DELAY]

  At Layer2:
    ∇_z2 L = ∇_y2 L * σ'(z2)               [needs B3 result!]

    B2: ∇_y1 L = W2^T @ ∇_z2 L             [NEEDED by Layer1 - CRITICAL
PATH]
    W2: ∇_W2 L = ∇_z2 L @ y1^T             [only for optimizer - CAN DELAY]

  At Layer1:
    ∇_z1 L = ∇_y1 L * σ'(z1)               [needs B2 result!]
```

```
    B1: ∇_x L = W1^T @ ∇_z1 L            [for previous stage - CRITICAL
PATH]
    W1: ∇_W1 L = ∇_z1 L @ x^T            [only for optimizer - CAN DELAY]

DEPENDENCY CHAIN:
==================

CRITICAL PATH (must be sequential):
  Loss → B3 → B2 → B1

WEIGHT GRADIENTS (can be done anytime after B):
  W3: depends on B3, but doesn't block B2
  W2: depends on B2, but doesn't block B1
  W1: depends on B1, but doesn't block anything

TIMING EXAMPLE - Standard Schedule:
====================================

Time | GPU1        | GPU2        | GPU3
-----|-------------|-------------|-------------
1    | F1          | idle        | idle
2    | F2          | F1          | idle
3    | F3          | F2          | F1          ← F1 completes, loss
calculated
4    | idle        | F3          | B1+W1       ← B1 STARTS on GPU3!
5    | idle        | B1+W1       | B2+W2       ← B1 flows to GPU2
6    | B1+W1       | B2+W2       | B3+W3       ← B1 reaches GPU1
7    | B2+W2       | B3+W3       | idle
8    | B3+W3       | idle        | idle

Bubble time: 2 idle steps per GPU (at start and end)

CRITICAL: Backward flows in REVERSE order!
- F1 path: GPU1 → GPU2 → GPU3
- B1 path: GPU3 → GPU2 → GPU1 (reverse!)

TIMING EXAMPLE - Optimized Schedule (ZB-H2):
=============================================

Time | GPU1        | GPU2        | GPU3
-----|-------------|-------------|-------------
1    | F1          | idle        | idle
2    | F2          | F1          | idle
3    | F3          | F2          | F1          ← F1 completes
4    | idle        | F3          | F2          ← F2 must flow through!
5    | idle        | idle        | F3          ← F3 completes, all
forwards done
6    | idle        | idle        | B3 (only!)  ← B3 starts on GPU3 (no
W!)
7    | idle        | B3          | B2          ← Backwards flow GPU3→
GPU2→GPU1
```

```
8    | B3          | B2          | B1          ← All B's flowing
9    | B2          | B1          | W3(MB1)     ← GPU3 does W3 for MB1
10   | B1          | W2(MB1)     | W3(MB2)     ← Each GPU does its own W!
11   | W1(MB1)     | W2(MB2)     | W3(MB3)     ← All GPUs doing their
W's
12   | W1(MB2)     | W2(MB3)     | idle
13   | W1(MB3)     | idle        | idle
```

Key improvements compared to standard schedule:
- B operations complete faster (separated from W)
- W operations scheduled to fill bubble time slots
- **CRITICAL: Each GPU computes W ONLY for its own layer!**
  - GPU1 computes W1 (for Layer1)
  - GPU2 computes W2 (for Layer2)
  - GPU3 computes W3 (for Layer3)
- W's computed for all micro-batches (MB1, MB2, MB3)
- B operations flow in reverse: GPU3 → GPU2 → GPU1
- W operations fill idle slots during cooldown phase

CRITICAL FLOW:
- Forwards: F1, F2, F3 all flow GPU1 → GPU2 → GPU3
- Backwards: B3, B2, B1 all flow GPU3 → GPU2 → GPU1
- Weights: Each GPU computes W for its OWN layer only
  - W cannot cross GPUs (weights/activations are local!)
- F1 done at t3, F2 done at t4, F3 done at t5
- Then backwards: B3(t6-8), B2(t7-9), B1(t8-10)
- Then weights: W3(t9-11), W2(t10-12), W1(t11-13)

CONCRETE NUMERICAL EXAMPLE:
==========

Assume:
- Forward time (F): t_f = 1ms
- Backward-B time: t_b = 0.5ms (activation gradients)
- Backward-W time: t_w = 0.5ms (weight gradients)
- Total backward: t_b + t_w = 1ms (same as before)

Standard 1F1B schedule:
  - Each step: F or (B+W), each takes 1ms
  - 3 GPUs, so bubble = (3-1) × 1ms = 2ms per GPU
  - Total bubble time: 2ms × 3 = 6ms

Optimized schedule (separate B and W):
  - Critical path: F (1ms) or B (0.5ms)
  - W operations (0.5ms) can overlap with bubbles
  - Bubble time with B only: (3-1) × 0.5ms = 1ms per GPU
  - The 2ms of W work can fit in the 1ms bubble! (some W extends past)
  - Net effect: ~50% bubble reduction for this example

WHY THIS WORKS:
==========

1. INDEPENDENCE: W doesn't produce outputs needed by other layers

   - B3's output (∇_y2 L) is needed by Layer2
   - W3's output (∇_W3 L) is only needed by optimizer

2. FLEXIBILITY: W can be done ANYTIME between B and optimizer.step()
   - Must happen after B of same layer (needs ∇_z L and activations)
   - Must happen before optimizer.step() (to have gradients ready)
   - Everything in between is fair game!

3. BUBBLE FILLING: Pipeline bubbles = idle GPU time
   - In standard 1F1B: GPUs idle at start (warmup) and end (cooldown)
   - With B/W split: Schedule W operations during these idle times
   - Result: Better GPU utilization, less wasted time

REAL IMPLEMENTATION EXAMPLE:
============================

Standard PyTorch backward:
```python
# Forward
y1 = layer1(x)
y2 = layer2(y1)
y3 = layer3(y2)
loss = criterion(y3, target)

# Backward (does B+W together)
loss.backward()  # Computes all ∇_W and ∇_y in one go
optimizer.step()
```

Conceptual split (pseudo-code):
```python
# Forward
y1 = layer1(x)
y2 = layer2(y1)
y3 = layer3(y2)
loss = criterion(y3, target)

# Backward - B phase (activation gradients, critical path)
grad_y3 = compute_loss_gradient(loss)
grad_y2 = layer3.backward_activations(grad_y3)  # B3
grad_y1 = layer2.backward_activations(grad_y2)  # B2
grad_x  = layer1.backward_activations(grad_y1)  # B1

# Backward - W phase (weight gradients, can be delayed)
# These can be scheduled flexibly!
layer3.backward_weights(grad_y3, y2)  # W3 - can do later
layer2.backward_weights(grad_y2, y1)  # W2 - can do later
layer1.backward_weights(grad_y1, x)   # W1 - can do later

# Optimizer (needs all W's done by now)
optimizer.step()
```

APPLYING TO PIPELINE PARALLELISM:

==========

Without B/W split:
  GPU1: F1 → F2 → (B1+W1) → (B2+W2) → optimizer.step()
  Each backward is (B+W) = longer critical path

With B/W split:
  GPU1: F1 → F2 → B1 → B2 → W1 → W2 → optimizer.step()
        ↑   ↑
        Critical path   Can fill bubbles!
        (needs to be fast)   (flexible scheduling)

The B operations form the critical path, while W operations can be strategically
placed to overlap with pipeline bubbles, achieving better GPU utilization.

SUMMARY:
========
- B (activation gradients): Critical path, blocks other layers, do ASAP
- W (weight gradients): Only for optimizer, flexible scheduling
- ZB-H2 schedule: Do all B's first (fast critical path), then fill bubbles with W's
- Result: Reduced pipeline bubble time without changing computation

KEY INSIGHTS
==========

1. Interleaving splits model layers into v chunks per GPU in a non-contiguous pattern
2. Each micro-batch visits each GPU v times (once per chunk owned by that GPU)
3. Smaller chunks → faster pipeline fill → reduced bubble overhead
4. Bubble ratio inversely proportional to v: doubling v halves the bubble
5. Trade-off: More communication overhead (data passes through GPUs multiple times)
6. Formula simplicity: bubble ratio depends only on p, v, m (not actual layer sizes!)

==========

TOPIC 4: DUALPIPE - BIDIRECTIONAL PIPELINE PARALLELISM

==========

::: Code Generated by Copilot [f9d8c1e2-4a7b-4c3d-8e9f-1a5b2d3c4e5f].
This comment will be removed automatically after the file is saved :::

CORE CONCEPT: Launch micro-batches from BOTH ENDS of the pipeline!
==========

Standard 1F1B Problem:
  - All micro-batches enter from GPU1 (first stage)
  - During warmup: Last GPU (GPU_p) sits IDLE
  - During cooldown: First GPU (GPU1) sits IDLE
  - Result: Wasted GPU cycles = bubble time

DualPipe Solution:
  - Split micro-batches into TWO STREAMS
  - Stream A: Enters from GPU1 (normal direction)
  - Stream B: Enters from GPU_p (reverse direction)
  - Both streams interleave to keep ALL GPUs busy!

ANALOGY - Highway Traffic:
==========

Imagine a 4-toll-booth highway:

Standard 1F1B (cars enter from one end only):
  Time 1: [Car1] [ ] [ ] [ ] ← Booths 2,3,4 idle!
  Time 2: [Car2] [Car1] [ ] [ ] ← Booths 3,4 idle!
  Time 3: [Car3] [Car2] [Car1] [ ] ← Booth 4 idle!
  Time 4: [Car4] [Car3] [Car2] [Car1] ← Finally all busy!

DualPipe (cars enter from BOTH ends):
  Time 1: [Car1→] [ ] [ ] [←Car8] ← 2 busy from start!
  Time 2: [Car2→] [Car1→] [←Car8] [←Car9] ← More busy!
  Time 3: [Car3→] [Car2→] [Car1→+←Car8] [←Car10] ← Even better!

Result: Tollbooths (GPUs) are utilized faster!

DETAILED EXAMPLE: 4 GPUs, Simple Case
==========

Model: 4 layers distributed across 4 GPUs
  GPU1: owns Layer1
  GPU2: owns Layer2
  GPU3: owns Layer3
  GPU4: owns Layer4

Micro-batches:
  Stream A (→): Batches A1, A2, A3 (enter from GPU1)
  Stream B (←): Batches B1, B2, B3 (enter from GPU4)

CRITICAL QUESTION: How can B1 enter from GPU4?
==========

GPU4 owns Layer4 (the LAST layer). You can't start processing at Layer4!

ANSWER - Two Possible Interpretations:

Interpretation 1: VIRTUAL REMAPPING

For "backward direction" batches, we REVERSE the GPU→Layer mapping:

Stream A batches (→):
  GPU1: Layer1 → GPU2: Layer2 → GPU3: Layer3 → GPU4: Layer4

Stream B batches (←):
  GPU4: Layer1 → GPU3: Layer2 → GPU2: Layer3 → GPU1: Layer4

This means:
- B1 enters GPU4, which processes it through Layer1
- Then B1 goes to GPU3 for Layer2
- Then B1 goes to GPU2 for Layer3
- Finally B1 goes to GPU1 for Layer4

The PHYSICAL layer order (1→2→3→4) is maintained, but the GPU assignment
is reversed for the backward stream!

Interpretation 2: SYMMETRIC ARCHITECTURE

For certain models (bidirectional transformers, autoencoders), you can process data in EITHER direction through the architecture:

Normal: Input → Encoder1 → Encoder2 → Encoder3 → Encoder4 → Output
Reverse: Input → Decoder4 → Decoder3 → Decoder2 → Decoder1 → Output

The "backward direction" uses a different but symmetric path through the model.

DETAILED TIMELINE USING INTERPRETATION 1:
==========

Setup: 4 GPUs, Stream A (A1, A2) and Stream B (B1, B2)

Stream A mapping: GPU1=L1, GPU2=L2, GPU3=L3, GPU4=L4 (normal)
Stream B mapping: GPU4=L1, GPU3=L2, GPU2=L3, GPU1=L4 (reversed)

Legend:
  F_A1(L1) = Forward pass of batch A1 through Layer1
  F_B1(L1) = Forward pass of batch B1 through Layer1
  B_A1(L1) = Backward pass of batch A1 through Layer1

CRITICAL: Stream A flows GPU1→GPU2→GPU3→GPU4
          Stream B flows GPU4→GPU3→GPU2→GPU1 (REVERSE!)

Time | GPU1 | GPU2 | GPU3 | GPU4

```
-----|-------------------|-------------------|-------------------|-------
------------
  1  | F_A1(L1)→         | idle              | idle              | ←
F_B1(L1)
     |                   |                   |                   |
  2  | F_A2(L1)→         | F_A1(L2)→         | ←F_B1(L2)         | ←
F_B2(L1)
     |                   |                   |                   |
  3  | idle              | F_A2(L2)→         | F_A1(L3)→         | idle
     |                   | ←F_B1(L3)         | ←F_B2(L2)         |
     |                   |                   |                   |
  4  | ←F_B1(L4)         | ←F_B2(L3)         | F_A2(L3)→         |
F_A1(L4)→
     |                   |                   |                   |
  5  | ←F_B2(L4)         | idle              | idle              |
F_A2(L4)→
     |                   |                   |                   |
B_A1(L4)← [bwd]
     |                   |                   |                   |
  6  | B_A1(L1)←         | ←B_B1(L3)         | B_A1(L3)←         |
B_A2(L4)← [bwd]
     | ←B_B1(L4) [bwd]   |                   | ←B_B1(L2) [bwd]   | ←
B_B1(L1) [bwd]
     |                   |                   |                   |
  7  | B_A2(L1)←         | B_A1(L2)←         | B_A2(L3)←         | ←
B_B2(L1) [bwd]
     | ←B_B2(L4) [bwd]   | ←B_B2(L3)         | ←B_B2(L2) [bwd]   |
```

```
STEP-BY-STEP TRACE:
===================

Time 1:
-------
GPU1: A1 enters, processes Layer1
GPU4: B1 enters, processes Layer1
Result: Both ends working simultaneously!

Time 2:
-------
GPU1: A2 enters, processes Layer1
GPU2: A1 (from Stream A) processes Layer2
GPU3: B1 (from Stream B) processes Layer2 ← Stream B goes GPU4→GPU3!
GPU4: B2 enters, processes Layer1
Result: All 4 GPUs busy!

Time 3:
-------
GPU1: Idle (waiting for B stream to arrive)
GPU2: A2 processes L2, B1 processes L3 (TWO batches!)
GPU3: A1 processes L3, B2 processes L2 (TWO batches!)
GPU4: Idle (waiting for next batch or backward)
```

```
Result: Middle GPUs at maximum load!

Time 4:
-------
GPU1: B1 arrives from Stream B for L4 processing
GPU2: B2 processes L3 (Stream B continues)
GPU3: A2 processes L3 (Stream A continues)
GPU4: A1 finishes forward pass with L4!
Result: A1 forward complete, backward can start

Time 5:
-------
GPU1: B2 arrives for L4 processing
GPU2: Idle
GPU3: Idle
GPU4: A2 finishes L4, A1 starts backward through L4
Result: Backward passes beginning

Time 6+:
--------
Backward passes flow in reverse:
- A1 backward: GPU4→GPU3→GPU2→GPU1
- B1 backward: GPU1→GPU2→GPU3→GPU4

COMPLETE FLOW PATHS:
====================

Batch A1 (Stream A, →):
  t1: GPU1 (L1) →
  t2: GPU2 (L2) →
  t3: GPU3 (L3) →
  t4: GPU4 (L4) → Forward done!
  t5-6: Backward GPU4→GPU3→GPU2→GPU1

Batch B1 (Stream B, ←):
  t1: GPU4 (L1) ←
  t2: GPU3 (L2) ←
  t3: GPU2 (L3) ←
  t4: GPU1 (L4) ← Forward done!
  t5-6: Backward GPU1→GPU2→GPU3→GPU4

Batch A2 (Stream A, →):
  t2: GPU1 (L1) →
  t3: GPU2 (L2) →
  t4: GPU3 (L3) →
  t5: GPU4 (L4) → Forward done!
  t6-7: Backward GPU4→GPU3→GPU2→GPU1

Batch B2 (Stream B, ←):
  t2: GPU4 (L1) ←
```

  t3: GPU3 (L2) ←
  t4: GPU2 (L3) ←
  t5: GPU1 (L4) ← Forward done!
  t6-7: Backward GPU1→GPU2→GPU3→GPU4

Key Pattern Recognition:
========================

Notice at Time 3:
- GPU2 needs to handle: F_A2(L2) AND F_B1(L3)
- GPU3 needs to handle: F_A1(L3) AND F_B2(L2)
- **CRITICAL: These happen SEQUENTIALLY, not simultaneously!**

What Actually Happens at Time 3:
- Time 3a: GPU2 processes F_A2(L2), GPU3 processes F_A1(L3)
- Time 3b: GPU2 processes F_B1(L3), GPU3 processes F_B2(L2)
- Result: Time slot 3 is TWICE as long as other time slots!

GPU Utilization (CORRECTED):
- Time 1: 2/4 GPUs busy (50%), 2 operations total
- Time 2: 4/4 GPUs busy (100%), 4 operations total
- Time 3: 2/4 GPUs busy (50%), but time slot is 2x longer to fit 4 operations
  - This is the "overlapped computation" bottleneck!
  - Middle GPUs become the limiting factor
- Time 4: 3/4 GPUs busy (75%), 3 operations total
- Time 5: 2/4 GPUs busy (50%), 2 operations total

KEY INSIGHT - The Load Imbalance Problem:
===

Middle GPUs are the bottleneck!
- GPU1 processes: A1, A2 (2 batches)
- GPU2 processes: A1, B1, A2, B2 (4 batches!) ← **2x the work**
- GPU3 processes: A1, B1, A2, B2 (4 batches!) ← **2x the work**
- GPU4 processes: B1, B2 (2 batches)

This means:
- Time slot 3 takes 2x longer than time slot 1
- Overall pipeline speed is limited by middle GPUs
- Trade-off: Less bubble time BUT longer per-batch latency

Throughput vs Latency:

✓ **Throughput IMPROVES**: More batches processed per unit time
✗ **Latency INCREASES**: Each individual batch takes longer (waiting in queue)

DualPipe optimizes for THROUGHPUT, not latency!

Compare to standard 1F1B:
- Time 1-3: Only 1-3 GPUs busy (GPU4 idle until time 4)

- DualPipe: All GPUs start working sooner, but middle GPUs overloaded
- Net effect: Better overall throughput, but individual batch latency may be worse

KEY OBSERVATIONS:
==========

1. SIMULTANEOUS START (Time 1):
   - GPU1 processes A1 through Layer1
   - GPU4 processes B1 through Layer1 (same layer, different GPU!)
   - Both ends working from the start!

2. MIDDLE GPUs ARE BUSIEST (Time 3):
   - GPU2 must process: F_A2(L2) AND F_B1(L3)
   - GPU3 must process: F_A1(L3) AND F_B2(L2)
   - **These operations happen SEQUENTIALLY (one after another)**
   - Time slot 3 takes 2x as long as a normal time slot
   - This is "overlapped" in the sense that both streams use the same GPU
   - Middle GPUs become the BOTTLENECK

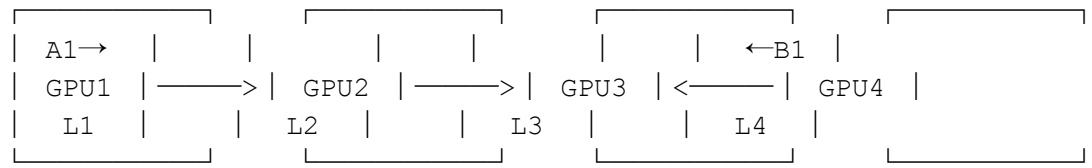
3. STREAMS CROSS IN THE MIDDLE:
   - Stream A flows: GPU1 → GPU2 → GPU3 → GPU4
   - Stream B flows: GPU4 → GPU3 → GPU2 → GPU1
   - They meet at middle GPUs (GPU2, GPU3)
   - Middle GPUs must handle double the workload

4. REDUCED BUBBLE TIME (but longer individual latency):
   - Compare to standard 1F1B where GPU4 is idle for 3 time steps
   - DualPipe: GPU4 starts working immediately (processing B1)
   - Similarly GPU1 doesn't idle during cooldown (processing B-stream)
   - **BUT**: Middle GPUs take longer per time slot
   - Result: Better aggregate throughput, possibly worse per-batch latency

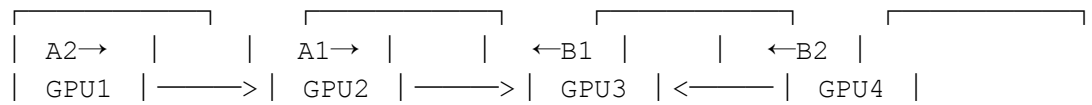
VISUAL DIAGRAM - DATA FLOW:
==========

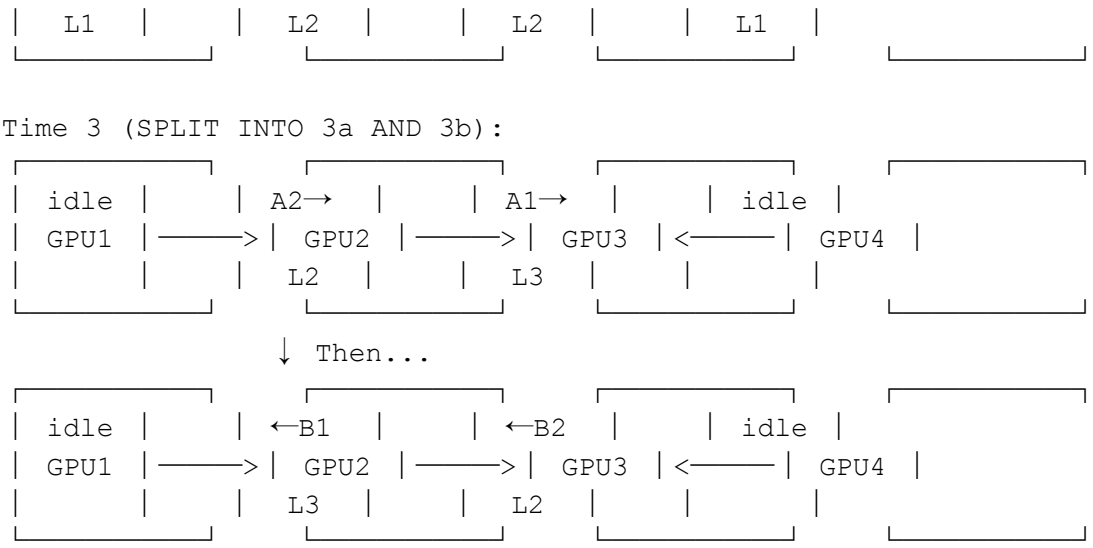
Stream A (→) flows left to right:
Stream B (←) flows right to left:

Time 1:



Time 2:





BUBBLE TIME ANALYSIS:
=====================

Standard 1F1B with 4 GPUs:
- Warmup bubble: 3 time steps (GPU4 idle)
- Cooldown bubble: 3 time steps (GPU1 idle)
- Total bubble per stream: (p-1) = 3 steps

DualPipe with 4 GPUs:
- Warmup: GPU4 processes Stream B (not idle!)
- Cooldown: GPU1 processes Stream B (not idle!)
- Bubble reduced significantly!
- Middle GPUs work harder (process both streams)
- Net effect: Bubble approaches ZERO in ideal case

Formula (from ZeroBubble paper):
With perfect scheduling via ILP optimization:
Bubble_ratio → 0% (theoretical "zero bubble")

PRACTICAL CONSIDERATIONS:
=========================

1. LOAD IMBALANCE (CRITICAL ISSUE):
   - Middle GPUs process ~2x micro-batches compared to edge GPUs
   - **They process batches SEQUENTIALLY, not simultaneously**
   - Time slots with overlapped work take 2x longer
   - This creates a bottleneck - middle GPUs limit throughput
   - Solutions:
     * Use smaller batch sizes for middle GPUs
     * Assign fewer layers to middle GPUs
     * Optimize scheduling with ILP solver

   - Trade-off: Reduced bubble but increased per-batch latency

2. MEMORY OVERHEAD:
   - Middle GPUs store activations for BOTH streams
   - Memory usage increases significantly (~2x)
   - Must have sufficient VRAM
   - Trade-off: More memory for less bubble time

3. SCHEDULING COMPLEXITY:
   - Finding optimal schedule is NP-hard
   - Solved using Integer Linear Programming (ILP)
   - Must respect dependencies between F, B, and W operations
   - Must balance load across GPUs
   - Pre-computed schedules for common configurations
   - Runtime overhead for complex schedules

4. LAYER ASSIGNMENT:
   - Stream B needs "reversed" GPU assignment OR
   - Symmetric model architecture
   - Not all models support this pattern
   - May need to duplicate/remap layers

5. THROUGHPUT vs LATENCY:
   - DualPipe optimizes for THROUGHPUT (batches/second)
   - Individual batch LATENCY may increase (due to queuing)
   - Good for training (aggregate throughput matters)
   - Bad for inference (per-request latency matters)

WHEN DOES DUALPIPE HELP MOST?
==============================

Best scenarios:
  ✓ Large number of pipeline stages (p >> 4)
  ✓ Many micro-batches (m >> p)
  ✓ Model supports bidirectional processing OR
  ✓ Willing to duplicate small layers for reversed mapping
  ✓ Sufficient GPU memory for overlapped processing

Limited benefit:
  ✗ Few pipeline stages (p = 2-4)
  ✗ Few micro-batches (m ≈ p)
  ✗ Memory-constrained scenarios
  ✗ Models with strict unidirectional dependencies

COMPARISON SUMMARY:
===================

Schedule | Bubble Time | Memory | Complexity | GPU
Utilization
-------------|--------------------|---------|------------|----------------
--

Standard 1F1B| (p-1)(t_f+t_b) | 1x | Simple | Low
(warmup/cooldown idle)
Interleaved | (p-1)(t_f+t_b)/v | 1.5x | Medium | Better (smaller
chunks)
B/W Split | (p-1)t_b | 1.5x | Medium | Better (W fills
bubbles)
DualPipe | ~0 (theoretical) | 2x | High (ILP) | Best (both
directions)

EXAMPLE FROM FIGURE:
====================

The attached figure shows:
  - 8 PP ranks (GPUs/devices)
  - 20 micro-batches total
  - Split into two streams (10 each direction)
  - Complex interleaving pattern
  - Colors show:
    • Orange: Forward pass
    • Dark Green: Backward pass (activation grads)
    • Light Green: Backward for input
    • Blue: Backward for weights
    • Orange-Green: Overlapped F & B computation

Notice:
  - Device 0 (GPU1): Starts immediately with batch 0
  - Device 7 (GPU8): Starts immediately with batch from other stream
  - Middle devices (3,4): Show heavy overlapping (multiple colors per
time)
  - Very little white space (idle time) = minimal bubble!

The schedule is computed by ILP solver to minimize total bubble time
while
respecting all computational dependencies.

IMPLEMENTATION NOTES:
=====================

DualPipe requires:
1. Splitting dataset into two halves (forward & backward streams)
2. Computing optimal schedule (offline, using ILP solver)
3. Executing schedule (runtime scheduler follows pre-computed plan)
4. Gathering results from both streams
5. Computing final gradients (accumulate from all micro-batches)

Code complexity is HIGH - not recommended unless:
  - You have many GPUs (p >= 8)
  - Bubble time is critical bottleneck
  - Memory budget allows 2x overhead
  - Team has expertise in distributed systems

KEY INSIGHTS

==

1. Interleaving splits model layers into v chunks per GPU in a non-contiguous pattern
2. Each micro-batch visits each GPU v times (once per chunk owned by that GPU)
3. Smaller chunks → faster pipeline fill → reduced bubble overhead
4. Bubble ratio inversely proportional to v: doubling v halves the bubble
5. Trade-off: More communication overhead (data passes through GPUs multiple times)
6. Formula simplicity: bubble ratio depends only on p, v, m (not actual layer sizes!)
7. B/W decomposition: Separate activation gradients (critical) from weight gradients (flexible)
8. DualPipe: Bidirectional streams reduce bubble to near-zero but require 2x memory and complex scheduling

PROGRESSIVE OPTIMIZATION PATH:
==
Stage 1: Standard 1F1B → Bubble = (p-1)(t_f+t_b)
Stage 2: Interleaved (v=2) → Bubble = (p-1)(t_f+t_b)/2 [50% reduction]
Stage 3: B/W Split → Bubble = (p-1)t_b [~50% more reduction]
Stage 4: DualPipe + ILP → Bubble → 0 [theoretical zero]

Each stage adds complexity but further reduces wasted GPU time!

==
END OF DOCUMENT
==